Labo 1 Geavanceerde computerarchitectuur

Gust Brouckaert - Tim Depickere

2024-11-06

1 Introductie

Voor het eerste labo werd er een array geflipt door te programmeren in CUDA die rechtstreeks op de GPU werkt. Om dit te realiseren moest er memory op de GPU gealloceerd worden en de data van de CPU overbrengen naar de GPU en omgekeerd voor de output.

2 GPU gebruik

Om ervoor te zorgen dat we de array flip op de GPU kunnen doen moet er eerst ruimte gealloceerd worden op de GPU, hierna zal de input naar deze ruimte worden gekopieerd.
Hiervoor werden de functies cudaMalloc en cudaMemcpy opgeroepen van de cuda_runtime library. We geven het argument cudaMemcpyHostToDevice mee met de cudaMemcpy zodat de functie weet dat er van de CPU naar de GPU gekopieerd wordt. Voor het kopieren van de output naar de input zullen we later het argument cudaMemcpyDeviceToHost meegegeven zodat de functie weet dat de src een pointer naar de GPU memory is.

Hierna werd de grid geinitialiseerd, wij kozen voor een blockSize van 256 en de gridSize afhankelijk van de size van onze inputArray. Hierdoor kunnen we garanderen dat er altijd voldoende threads zullen aangemaakt worden. Je kan de blocksize zelf kiezen natuurlijk. Hierna zullen we onze flipArray functie oproepen.

Tenslotte kopieren we onze output terug naar onze inputArray op de CPU en dealloceren we de memory op de GPU met behulp van de cudaFree functie.

```
int *outputArray;
//allocate room for the outputArray on the GPU
cudaMalloc((void**)&outputArray, sizeof(int) * size);
//copy the inputArray into the outputArray
cudaMemcpy(outputArray, inputArray, sizeof(int) * size, cudaMemcpyHostToDevice);

dim3 blockSize(256);
dim3 gridSize((size + blockSize.x - 1) / blockSize.x);

flipArray<<<gridSize, blockSize>>>(outputArray, size);
//copy the outputArray into the inputArray
cudaMemcpy(inputArray, outputArray, sizeof(int) * size, cudaMemcpyDeviceToHost);
```

3 FlipArray

De flipArray functie is redelijk eenvoudig, we splitsen onze Array in 2 en wisselen dan de elementen onderling. Om ervoor te zorgen dat we correcte indexing van onze threads doen wordt eerst de idx berekent. Hierdoor zullen de threads nog steeds met de correcte input waarde werken als er meerdere blocks in de grid zijn.

```
__global__ void flipArray(int *array, int size) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < size / 2) {
        int temp = array[idx];
        array[idx] = array[size - idx - 1];
        array[size - idx - 1] = temp;
    }
}
```